HDOC Day-65

Tower of Hanoi Using Recursion in C

Aim

To write a recursive C program that solves the Tower of Hanoi problem for n disks, displays all moves from source rod A to destination rod C using auxiliary rod B, and calculates the total number of moves.

Algorithm

1. Start the program and read the number of disks n.
2. Validate that n is positive and at most 63; otherwise display an error.
3. Define towerOfHanoi(n, source, auxiliary, destination).
4. If n equals 1, display the move from source to destination and return.
5. Otherwise, recursively move n - 1 disks from source to auxiliary using destination.
6. Display the move of disk n from source to destination.
7. Recursively move n - 1 disks from auxiliary to destination using source.
8. Calculate the minimum moves using 2^n - 1, display the result, and stop.

C Program

```
#include <stdio.h>

void towerOfHanoi(int n, char source,
                  char auxiliary, char destination);

int main()
{
    int n, i;
    unsigned long long totalMoves = 1;

    printf("Enter the number of disks: ");
    scanf("%d", &n);

    if (n <= 0 || n > 63)
    {
        printf("Invalid number of disks! Enter 1 to 63.\n");
        return 0;
    }

    printf("\nTower of Hanoi Moves:\n");
    towerOfHanoi(n, 'A', 'B', 'C');

    for (i = 0; i < n; i++)
        totalMoves *= 2;
    totalMoves = totalMoves - 1;

    printf("\nTotal Moves: %llu\n", totalMoves);
    return 0;
}

void towerOfHanoi(int n, char source,
                  char auxiliary, char destination)
{
    if (n == 1)
    {
        printf("Move disk 1 from %c to %c\n",
               source, destination);
        return;
    }

    towerOfHanoi(n - 1, source,
                 destination, auxiliary);

    printf("Move disk %d from %c to %c\n",
           n, source, destination);

    towerOfHanoi(n - 1, auxiliary,
                 source, destination);
}
```

Explanation

The puzzle has three rods: A (source), B (auxiliary), and C (destination). Only one disk may be moved at a time, and a larger disk cannot be placed on a smaller disk. The base case moves one disk. For n disks, recursion first transfers n - 1 disks to the auxiliary rod, then moves the largest disk, then transfers the n - 1 disks onto the destination rod.

The recurrence is T(n) = 2T(n - 1) + 1, giving the minimum number of moves T(n) = 2^n - 1. Time complexity is O(2^n); recursive stack space is O(n).

Test Case Table

Test case	Number of disks	Expected moves
1	1	1
2	2	3
3	3	7
4	4	15
5	5	31
6	6	63
7	7	127
8	8	255
9	0	Invalid input
10	-3	Invalid input

Sample Outputs

Three disks

```
Enter the number of disks: 3

Tower of Hanoi Moves:
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C

Total Moves: 7
```

Two disks

```
Enter the number of disks: 2

Tower of Hanoi Moves:
Move disk 1 from A to B
Move disk 2 from A to C
Move disk 1 from B to C

Total Moves: 3
```

Single disk

```
Enter the number of disks: 1

Tower of Hanoi Moves:
Move disk 1 from A to C

Total Moves: 1
```

Invalid input

```
Enter the number of disks: 0
Invalid number of disks! Enter 1 to 63.
```

Result

The recursive Tower of Hanoi program displays all legal moves needed to transfer n disks from source to destination using an auxiliary rod. It computes the minimum move count as 2^n - 1 and handles invalid disk counts.